

Theme 1 — AI-Driven Parking Intelligence

Full External Asset Map + Execution Plan (Flipkart Grid 2026)

PART A — What you actually have

Your dataset: `jan_to_may_police_violation...csv` — 298,450 rows, Bengaluru Traffic Police records, Jan-May.

Confirmed columns and what they give you:

- `latitude`, `longitude`, `location` — real GPS + reverse-geocoded address per violation
- `vehicle_number`, `vehicle_type` — SCOOTER (94.8k), CAR (88.9k), MOTOR CYCLE (40.8k), PASSENGER AUTO (37.8k), MAXI-CAB, LGV, GOODS AUTO, etc.
- `violation_type` — JSON-array-as-string, dominated by `NO PARKING`, `WRONG PARKING`, `PARKING NEAR ROAD CROSSING`, `DOUBLE PARKING`, `PARKING IN A MAIN ROAD`, `DEFECTIVE NUMBER PLATE`
- `offence_code` — numeric codes paired positionally with `violation_type` array
- `created_datetime`, `closed_datetime`, `modified_datetime` — full timestamps, enables temporal analysis
- `police_station` (54 unique), `junction_name`, `center_code`, `device_id` — your spatial/organizational units
- `validation_status` (approved/rejected/processing/duplicate) — signal on data quality / false-positive rate
- `updated_vehicle_number`, `updated_vehicle_type` — correction trail, useful for an "ANPR error rate" angle

What's missing (confirmed gap, same gap the research doc flags as missing *everywhere*, not just for you):

- No traffic speed/flow/delay data
- No image/video — this is metadata only, no visual evidence
- No congestion-severity ground truth

This means: your core asset is real, large, structured violation telemetry. Your core *gap* is everything visual and everything about downstream traffic impact. The external assets below exist specifically to fill those two gaps.

PART B — Every external asset from the research doc, mapped to what it fills

B1. Datasets (fill the "no visual data" gap)

Dataset	What it actually gives you	Fills which gap	Priority
UVH-26 / VehicleNet-Y26s (HuggingFace, iisc-aim/UVH-26), (Perception365/VehicleNet-Y26s)	26,646 images from ~2,800 Bengaluru Safe-City CCTV cameras , 1.8M bounding boxes, 14 India-specific vehicle classes (2-wheeler, 3-wheeler, hatchback, sedan, SUV, etc.)	Visual layer — and it's literally your city's own CCTV network	Highest — verified live, currently trending #1 object-detection dataset on HF (confirmed Nov 2025 release)
PKLot (Voxel51/PKLot), HF)	12,416 images, ~695k boxes, occupancy classification, weather variation	Backup/supplementary occupancy-classifier training data if UVH-26 alone isn't enough	Medium
CCPD	~300k images, China, ANPR/OCR training	Only useful if you need to pretrain an OCR backbone from scratch — skip this, use Awiros instead (see B2)	Low — skip
UA-DETRAC	140k frames, MOT/tracking-focused, vehicle categories	Useful for DeepSORT tracking practice if you build a dwell-time-from-video demo	Medium, optional
NYC 311 Illegal Parking	Tabular violation data, NYC	Structurally identical to what you already have — useful only as a methodology reference for how to structure a violation→dispatch pipeline, not as data to merge	Reference only
CARPK, HighD, DroneVehicle	Drone/aerial vehicle datasets	Not relevant unless you want a "future: drone enforcement" slide — skip for core build	Skip

B2. Pretrained models (fill the "we need a visual detection layer fast" gap)

Model	Source	What it does	Reusability
VehicleNet-Y26s / Y26n	HuggingFace <code>Perception365/</code>	Multi-class (14 types) vehicle detector, trained on real Bengaluru CCTV (UVH-26). Apache 2.0 licensed.	Drop-in, zero training needed. This is your single best asset — it's literally trained on your city's camera network.
<code>nickmuchi/yolos-small-finetuned-license-plate-detection</code>	HuggingFace	YOLOS-based plate detector, 49.0 AP	High — instant via HF <code>pipeline()</code> API, skip building your own plate localizer
Generic YOLOv8/v11 (Ultralytics)	Open source	Base detector if you want to fine-tune on a restricted-zone polygon yourself	Use as fallback if VehicleNet doesn't generalize to your specific footage

B3. Algorithms / libraries (fill the "turn raw violations into insight" gap)

Tool	Source	What it does for you specifically
<code>st_dbscan</code> (<code>eren-ck/st_dbscan</code> , pip-installable)	GitHub/PyPI	Feed it <code>((latitude, longitude, created_datetime))</code> directly from your CSV. Outputs persistent chronic hotspots , filtering out one-off violations. This is the single highest-leverage, lowest-effort move available — <code>pip install st_dbscan</code> , a few lines of code, immediately differentiates you from anyone doing a naive heatmap.
Space Pressure Index (SPI) formula	From the doc (EWR × OL)	You don't have lane-width/vehicle-pixel data to compute this exactly — but you can build a defensible proxy version : substitute EWR with violation density per road segment, OL with average dwell time inferred from <code>created_datetime</code> → <code>closed_datetime</code> . Be explicit in your deck that this is a principled proxy, not measured EWR — judges respect honesty about scope.
Queueing theory (QL = N × Lv)	From the doc	Same proxy treatment — use violation <i>count</i> per junction per hour as a stand-in for N (stopped vehicles), since you don't have real-time vehicle counts.

Tool	Source	What it does for you specifically
LSTM / Quasi-Poisson regression	From the doc	Apply directly to your <code>created_datetime</code> field — you have 5 months of real timestamped violations, more than enough to fit a temporal pattern model (time-of-day, day-of-week, commercial-delivery-hour spikes). This is fully supported by your data with no proxy needed.

B4. Reference repos (fill the "system architecture" gap)

Repo	What to take from it
<code>benarnav/parking-monitor</code> (GitHub)	Fork the <i>architecture</i> , not necessarily the code: ROI-based detection → dwell-time tracking → dispatch-API payload. Use this as your system diagram's backbone even if your core analytics run on the CSV rather than live video.
<code>coding-brigade/advanced-anpr-fr</code>	Reference for evidence-packaging/database schema (timestamp + GPS + OCR crop → structured record) — your CSV already mirrors this schema, so this mainly validates that your data model is "production-shaped."
<code>prakharninja0927/illegal-Parking-Detection</code>	Skip — outdated YOLOv4, not worth forking over VehicleNet-Y26s.

PART C — Feature ideas from the doc, ranked by buildability against YOUR data in 4 days

Tier 1 — Build these (directly supported by your CSV, high judge impact)

- ST-DBSCAN Chronic Hotspot Map** — cluster violations by lat/long + time, output a ranked list of junctions/police-stations with *persistent* (not one-off) violation patterns. Visualize on a Leaflet/Folium map.
- Enforcement Prioritization Engine** — score each hotspot by (violation count × recency × violation-severity weight, e.g. DOUBLE PARKING > NO PARKING). Rank all 54 police stations. This is your core deliverable — directly answers "detect hotspots... enable targeted enforcement."
- Temporal Pattern Dashboard** — day-of-week / hour-of-day violation heatmap per top hotspot (fully supported by `created_datetime`, zero proxy needed). Strong, easy, visually clean.
- XAI / Explainability Layer (LLM-generated justification)** — for each top-ranked hotspot, auto-generate a sentence like *"Madiwala junction recorded 340 wrong-parking violations across 5 months, concentrated 6-9 AM weekdays — consistent with*

school/office-commute spillover." Cheap to build (one LLM call per hotspot), very high judge-perception impact, directly lifted from the doc's XAI section.

5. **Validation-status / data-quality angle** — your `validation_status` field (42% approved/rejected/etc.) is a free, real signal nobody else will think to use: show your system can flag likely-false-positive violation patterns by station, which is a genuinely novel addition not even in the research doc.

Tier 2 — Build if time allows (adds visual "wow factor" for the demo)

6. **Live CV Detection Demo using VehicleNet-Y26s** — run it on a short public Bengaluru traffic clip (or UVH-26 sample images) live during your pitch, showing "this is how new violations would be detected and fed into the same pipeline." This bridges your tabular analytics to a visual demo without needing to train anything.
7. **ANPR demo** — pair VehicleNet-Y26s output with `nickmuchi/yolos-small-finetuned-license-plate-detection` for a plate-localization demo on a sample image. Pure visual polish, not core logic.

Tier 3 — Mention in deck as "future work," do not attempt to build

8. Citizen crowdsourcing app, gamification/VPC tokens, Waze/Maps integration, equity-balancing algorithm, federated learning — all good **slide content** (shows research depth, "we thought about ethics/scale") but zero of these are buildable or demoable in 4 days. List them explicitly as a "Roadmap beyond hackathon" slide — judges like seeing scope-awareness.

PART D — Day-by-day plan (today = Day 0, deadline = 21st)

Day 1 — Data engineering + first hotspot output

- Fix `violation_type`/`offence_code` JSON-array-as-string parsing (the embedded `""..""` escaping needs cleanup before any analysis).
- Compute dwell-time proxy where possible (`closed_datetime` - `created_datetime`), handling the many NULLs in `closed_datetime`).
- `pip install st-dbscan`, run it on cleaned lat/long/timestamp → get first hotspot clusters.
- **Checkpoint:** by end of Day 1 you should have a list of ranked chronic hotspots, even if unstyled.

Day 2 — Scoring engine + temporal layer

- Build the enforcement prioritization score (violation count × severity weight × recency decay).

- Fit the temporal pattern layer (hour/day-of-week breakdown per top-N hotspots) — simple groupby/pivot is enough, LSTM is optional polish if time allows.
- Wire up the LLM explainability call for top 10 hotspots.
- **Checkpoint:** ranked, explained hotspot list with temporal context — this alone is a complete, defensible submission.

Day 3 — Visual layer + CV demo

- Build the map dashboard (Folium/Leaflet — plot hotspots, color by priority score, popup with LLM explanation).
- Pull a short Bengaluru traffic clip or sample UVH-26 images, run VehicleNet-Y26s inference live, screenshot/record the output for the deck or a live demo segment.
- Optional: chain in the license-plate model for a 10-second "this is what evidence generation would look like" clip.
- **Checkpoint:** working dashboard + a recorded or live CV demo clip as backup if live demo risks failing on stage.

Day 4 — Polish, deck, rehearsal

- Build the pitch deck: problem → real BTP data → ST-DBSCAN insight → prioritization engine → explainability → (CV demo) → roadmap (citizen app, equity, federated learning as future work).
- Rehearse a tight 2-3 minute narrative. Lead with "real Bengaluru Traffic Police data, 5 months, 298k records" — even though it was provided to everyone, very few teams will have done spatio-temporal clustering *and* built an explainability layer *and* shown a live CV demo on top of it.
- Have a fallback: if live CV demo is risky, pre-record it.

PART E — Why this combination should stand out

Most teams under time pressure will likely stop at "count violations, draw a heatmap." Your differentiation stack, in order of how non-obvious each move is:

1. ST-DBSCAN (persistent vs. one-off hotspots) — a meaningful step up from a naive heatmap, low effort.
2. Severity-weighted, recency-decayed prioritization score — turns raw counts into an actionable ranking, which is what the problem statement actually asks for ("enable targeted enforcement").
3. LLM-generated explainability per hotspot — directly addresses the "cities lack a system that explains the ticket's justification" gap the research doc explicitly calls out as missing industry-wide.
4. VehicleNet-Y26s tie-in — the fact that your visual-detection asset is trained on *the same city's CCTV network* as your tabular data is a coherent, defensible "this is

deployable in Bengaluru, not a generic demo" story.

5. The validation_status data-quality angle — genuinely not in the research doc at all, entirely your own observation from the real data, which is exactly the kind of original insight judges remember.

If you want, I can start writing the actual data-cleaning + ST-DBSCAN script next so you walk into Day 1 with working code instead of a blank notebook — just re-upload the police violation CSV since it's not currently sitting in this session's file system.